# Python Engineering - Class 05 Reference Guide

Listas, tuplas y trayectorias

Guía de consulta de sintaxis y herramientas utilizadas en esta clase. Incluye qué hace cada elemento, cómo se escribe, qué recibe o produce y cuándo conviene utilizarlo.

## Mapa rápido

| Concepto | Sintaxis | Idea |
|---|---|---|
| list | values = [] | Colección ordenada y mutable que puede crecer dinámicamente. |
| append() | values.append(x) | Añade un nuevo elemento al final de una lista. |
| Indexación | values[i] | Selecciona un elemento mediante su posición; Python empieza a contar desde 0. |
| Índices negativos | values[-1] | Permite seleccionar elementos desde el final; -1 representa el último. |
| Slicing | values[a:b] | Selecciona una subsecuencia sin modificar necesariamente la colección original. |
| tuple | state = (h,v,t) | Contenedor ordenado cuya estructura no puede reasignarse; aquí se usa como snapsho |
| Mutabilidad | list mutable / tuple container immutable | Una lista puede modificarse in-place; la estructura de una tuple no puede reasignarse. |
| Referencias compartidas / aliasing | a = b | Dos nombres pueden referirse al mismo objeto mutable, de modo que una modificación s |
| Expresión generadora | max(s[0] for s in trajectory) | Genera valores de forma perezosa para consumirlos en funciones como max(), min() o s |
| lambda | lambda s: s[0] | Función anónima breve, útil por ejemplo como criterio key=. |

## 1. list

**Sintaxis:** `values = []`

Colección ordenada y mutable que puede crecer dinámicamente.

```
trajectory = []
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 2. append()

**Sintaxis:** `values.append(x)`

Añade un nuevo elemento al final de una lista.

```
trajectory.append((h, v, t))
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 3. Indexación

**Sintaxis:** `values[i]`

Selecciona un elemento mediante su posición; Python empieza a contar desde 0.

```
heights = trajectory[:, 0]
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 4. Índices negativos

**Sintaxis:** `values[-1]`

Permite seleccionar elementos desde el final; -1 representa el último.

```
last_state = trajectory[-1]
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 5. Slicing

**Sintaxis:** `values[a:b]`

Selecciona una subsecuencia sin modificar necesariamente la colección original.

```
subset = values[1:4]
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 6. tuple

**Sintaxis:** `state = (h,v,t)`

Contenedor ordenado cuya estructura no puede reasignarse; aquí se usa como snapshot de estado.

```
state = (h, v, t)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 7. Mutabilidad

**Sintaxis:** `list mutable / tuple container immutable`

Una lista puede modificarse in-place; la estructura de una tuple no puede reasignarse.

```
values = [1, 2]
values[0] = 99
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 8. Referencias compartidas / aliasing

**Sintaxis:** `a = b`

Dos nombres pueden referirse al mismo objeto mutable, de modo que una modificación se observa desde ambos.

```
a = [1, 2]
b = a
b.append(3)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 9. Expresión generadora

**Sintaxis:** `max(s[0] for s in trajectory)`

Genera valores de forma perezosa para consumirlos en funciones como max(), min() o sum().

```
max_h = max(state[0] for state in trajectory)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## 10. lambda

**Sintaxis:** `lambda s: s[0]`

Función anónima breve, útil por ejemplo como criterio key=.

```
highest = max(trajectory, key=lambda state: state[0])
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

## Errores comunes

- Confundir el nombre de una función con su llamada usando paréntesis.
- No revisar tipos, índices o shapes antes de una operación.
- Usar una herramienta porque funciona sin poder explicar qué recibe y qué devuelve.

## Chuleta mental

```
list: values = []
append(): values.append(x)
Indexación: values[i]
Índices negativos: values[-1]
Slicing: values[a:b]
tuple: state = (h,v,t)
Mutabilidad: list mutable / tuple container immutable
Referencias compartidas / aliasing: a = b
Expresión generadora: max(s[0] for s in trajectory)
lambda: lambda s: s[0]
```